RAG-Based Customer Support Assistant using LangGraph & HITL

LOW LEVEL DESIGN (LLD)

Objective

The objective of this Low-Level Design document is to describe the internal implementation details of the RAG-Based Customer Support Assistant. This document explains module-level architecture, workflow execution, state management, routing logic, data structures, APIs, and error handling mechanisms required for practical implementation.

1. Module-Level Design

1.1 Document Processing Module

Functions

- Load PDFs
- Extract text
- Clean unwanted characters
- Handle malformed documents

Input

- PDF file

Output

- Raw extracted text

1.2 Chunking Module

Functions

- Split text into chunks
- Apply overlap
- Maintain semantic continuity

Input

- Extracted text

Output

- List of chunks

1.3 Embedding Module

Functions

- Generate vector embeddings
- Normalize vectors
- Store metadata

Input

- Text chunks

Output

- Numerical vectors

1.4 Vector Storage Module

Functions

- Store embeddings
- Perform similarity search
- Maintain persistent storage

Database

- ChromaDB

1.5 Retrieval Module

Functions

- Accept user query
- Generate query embedding
- Retrieve top-k relevant chunks

Output

- Relevant document context

1.6 Query Processing Module

Functions

- Combine retrieved chunks
- Generate prompts
- Send context to LLM

1.7 Graph Execution Module

Functions

- Execute graph nodes
- Maintain workflow state
- Route execution dynamically

1.8 HITL Module

Functions

- Detect escalation conditions
- Notify human support
- Integrate human response

2. Data Structures

Document Structure

Document = {

"id": str,

"text": str,

"metadata": dict }

Chunk Structure

Chunk = {

"chunk_id": str,

"content": str,

"page_number": int }

Embedding Structure

Embedding = {

"vector": list,

"chunk_id": str }

Query Response Schema

Response = {

"query": str,

"answer": str,

"confidence": float,

"source_chunks": list }

LangGraph State Object

State = {

"query": str,

"retrieved_docs": list,

"response": str,

"confidence": float,

"escalation_required": bool }

3. Workflow Design using LangGraph

Nodes

Processing Node

- Retrieves relevant chunks
- Sends context to LLM
- Generates response

Output Node

- Displays final response
- Handles escalation messages

Edges

Input Node -> Processing Node -> Output Node

Conditional Edge:

If confidence < threshold:

Route to HITL Node

Else:

Route to Output Node

State Management

State management is one of the core concepts in LangGraph workflows. The state object carries information between workflow nodes and ensures that every node has access to the latest query data, retrieved documents, confidence scores, and generated responses.

State Variables

state = {

"query": "user question",

"retrieved_docs": [],

"response": "",

"confidence": 0.0,

"escalation_required": False

}

Purpose of State

- Maintains workflow continuity
- Shares data between nodes
- Supports conditional routing
- Enables HITL escalation logic
- Stores intermediate processing results

4. Conditional Routing Logic

Answer Generation Criteria

- Relevant chunks found
- Confidence above threshold

- Context availability

Escalation Criteria

- Low confidence score
- Missing context
- Sensitive customer query
- Multi-step reasoning required

5. HITL Design

Escalation Trigger

The system escalates when:

- Confidence score < 0.6
- Retrieved context is insufficient
- Query contains unsupported requests

Escalation Workflow

User Query

↓

AI Processing

↓

Confidence Check

↓

Escalate to Human Agent

↓

Human Response

↓

Final User Response

6. API / Interface Design

Input Format

{

"query": "What is the refund policy?"

}

Output Format

{

"answer": "Refunds are processed within 7 business days.",

"confidence": 0.89,

"escalated": false

}

7. Error Handling

Missing Data

- Validate uploaded PDF
- Display upload errors

No Relevant Chunks Found

- Trigger clarification request
- Escalate if required

LLM Failure

- Retry mechanism
- Backup response handling